

EE216 Homework4

Introduction

Finite Impulse Response (FIR) filters are commonplace in digital signal processing (DSP) applications -- they are perhaps the most widely used operation in this domain. They are well suited for hardware implementation since they can be implemented as a highly optimized architecture. A key property is that they are a linear transform on contiguous elements of a signal. This maps well to data structures (e.g., FIFOs or tap delay lines) that can be implemented efficiently in hardware. In this lab, you are going to optimize the throughput of an FIR filter written in HLS.

Declaration of the top-level

```
#define N 16

typedef int coef_t;
typedef int data_t;
typedef int acc_t;

void fir(data_t *y, data_t x) {
    coef_t c[N] = {1,-2,3,-4,5,-6,7,-8,-8,7,-6,5,-4,3,-2,1};
    static data_t shift_reg[N];
    acc_t acc;
    int i;

    acc = 0;

    for (i = N - 1; i >= 0; i--) {

        if (i == 0) {
            acc += x * c[0];
            shift_reg[0] = x;
        } else {
            shift_reg[i] = shift_reg[i - 1];
            acc += shift_reg[i] * c[i];
        }
    }
    *y = acc;
}
```

The code given above is a functionally correct, but highly unoptimized implementation of a 15 tap FIR filter. The performance and utilization estimates are given below.

```
== Performance Estimates
+-----+-----+-----+-----+
| Latency | Interval |
| min | max | min | max |
+-----+-----+-----+-----+
| 36 | 36 | 37 | 37 |
+-----+-----+-----+-----+
```

== Utilization Estimates

Name	BRAM_18K	DSP48E	FF	LUT	URAM
DSP	-	-	-	-	-
Expression	-	-	0	78	-
FIFO	-	-	-	-	-
Instance	-	0	0	20	-
Memory	0	-	96	16	-
Multiplexer	-	-	-	105	-
Register	-	-	217	-	-
Total	0	0	313	219	0
Available	624	1728	460800	230400	96
Utilization (%)	0	0	~0	~0	0

Requirements

In this homework, you are required to:

1. Write a testbench for the function. The testbench should generate an impulse whose amplitude is one. Besides, you should generate a golden result to ensure the functionality of your optimized FIR filter is correct.
2. Try to optimize the performance of the FIR filter, which is measured by the interval. Besides, you should choose the zcu104 board and set the frequency to 200MHz when creating a project.
3. Perform Co-Simulation and generate the waveform.

Here is an example, after optimization, the performance is greatly improved, the interval is reduced to 1 but the utilization is also greatly increased.

== Performance Estimates

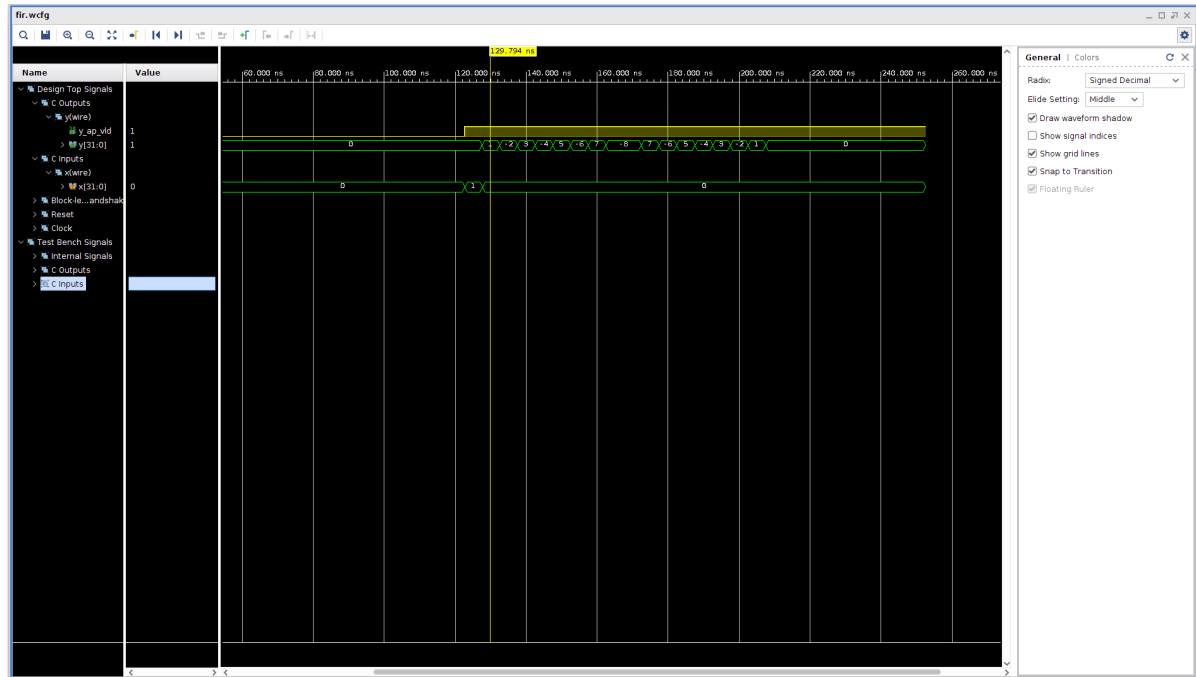
Latency	Interval
min	max
2	1

== Utilization Estimates

Name	BRAM_18K	DSP48E	FF	LUT	URAM
DSP	-	-	-	-	-
Expression	-	-	0	787	-
FIFO	-	-	-	-	-
Instance	-	-	-	-	-
Memory	-	-	-	-	-
Multiplexer	-	-	-	-	-
Register	-	-	899	-	-

Total		0	0	899	787	0
Available		624	1728	460800	230400	96
Utilization (%)		0	0	~0	~0	0

You can see from the waveform that the input of the FIR is an impulse, and the output is generated 1 cycle behind the input signal.



Submission: Deadline on 2025-11-27 23:59 PM

1. Code of the FIR filter.
2. Code of the testbench.
3. The generated waveform file.
4. A report describing your optimizing method. The screenshots of the result of C-Simulation, Synthesis, and Co-Simulation should be included in the pdf file.

Please compress all the files into a ZIP file with name **HW4_YourName_StudentNumber.zip**. The file structure should be like:

```

HW4_孟世杰_2022231101.zip
├─ FIR.cpp
├─ FIR_tb.cpp
├─ Report.pdf
├─ waveform.wcfg

```

Overdue Policy

- Submit on time, get all scores.
- Submit within 24 hours of the deadline, get half of the score.
- Submit more than 24 hours after the deadline, get 0 score.